
CIS 678 - Machine Learning

– Decision Trees –

— Instructor: Yong Zhuang —

yong.zhuang@gvsu.edu

Last Class

- Naïve Bayes
 - What is the Naïve Bayes assumption?
 - Why do we use it?
 - How do we learn the model?
 - Why is Bayesian estimation important?

Practice: Naïve Bayes Email Classification

Consider the following new email:

$\mathbf{x}=(\text{sender known=no,topic=work,contains link=no,urgent words=no})$.

1. Predict its class using MLE estimates.
2. Identify any zero-probability problem.
3. Repeat the prediction using add-one smoothing.



$$\begin{aligned}\hat{\pi}_k^{MLE} &= \hat{P}(Y = k) = \frac{N_k}{N} \\ \hat{\theta}_{i,v,k}^{MLE} &= \hat{P}(X_i = v \mid Y = k) = \frac{N_{i,v,k}}{N_k} \\ \hat{P}(\mathbf{X} = \mathbf{x} \mid Y = k) &= \prod_{i=1}^d \hat{P}(X_i = x_i \mid Y = k) \\ &= \prod_{i=1}^d \hat{\theta}_{i,x_i,k}^{MLE} = \prod_{i=1}^d \frac{N_{i,x_i,k}}{N_k}\end{aligned}$$

	ID	Sender known	Topic	Contains link	Urgent words	Class
	1	no	promotion	yes	yes	spam
	2	no	promotion	yes	no	spam
	3	no	account	yes	yes	spam
	4	no	work	yes	yes	spam
	5	yes	promotion	yes	no	spam
	6	no	account	no	yes	spam
	7	yes	account	yes	yes	spam
	8	yes	work	no	no	legitimate
	9	yes	account	no	no	legitimate
	10	yes	account	yes	no	legitimate
	11	yes	work	yes	no	legitimate
	12	yes	account	no	yes	legitimate
	13	yes	work	no	yes	legitimate
	14	yes	account	no	no	legitimate

Practice: Naïve Bayes Email Classification

$$\hat{\pi}_{spam}^{MLE} = \frac{N_{spam}}{N} = \frac{7}{14}$$
$$\hat{\pi}_{legitimate}^{MLE} = \frac{N_{legitimate}}{N} = \frac{7}{14}$$

$$\hat{\pi}_k^{MLE} = \hat{P}(Y = k) = \frac{N_k}{N}$$

$$\hat{\theta}_{i,v,k}^{MLE} = \hat{P}(X_i = v | Y = k) = \frac{N_{i,v,k}}{N_k}$$

$$\hat{P}(\mathbf{X} = \mathbf{x} | Y = k) = \prod_{i=1}^d \hat{P}(X_i = x_i | Y = k)$$
$$= \prod_{i=1}^d \hat{\theta}_{i,x_i,k}^{MLE} = \prod_{i=1}^d \frac{N_{i,x_i,k}}{N_k}$$

ID	Sender known	Topic	Contains link	Urgent words	Class
1	no	promotion	yes	yes	spam
2	no	promotion	yes	no	spam
3	no	account	yes	yes	spam
4	no	work	yes	yes	spam
5	yes	promotion	yes	no	spam
6	no	account	no	yes	spam
7	yes	account	yes	yes	spam
8	yes	work	no	no	legitimate
9	yes	account	no	no	legitimate
10	yes	account	yes	no	legitimate
11	yes	work	yes	no	legitimate
12	yes	account	no	yes	legitimate
13	yes	work	no	yes	legitimate
14	yes	account	no	no	legitimate

Practice: Naïve Bayes Email Classification

$\mathbf{x}=(\text{sender known=no,topic=work,contains link=no,urgent words=no})$

$$\begin{aligned}\hat{\theta}_{\text{sender-known=no,class=spam}}^{MLE} &= \frac{N_{\text{sender-known=no,class=spam}}}{N_{\text{spam}}} \\ &= \frac{5}{7} \\ \hat{\theta}_{\text{sender-known=no,class=legitimate}}^{MLE} &= \frac{N_{\text{sender-known=no,class=legitimate}}}{N_{\text{legitimate}}} \\ &= \frac{0}{7}\end{aligned}$$



$$\begin{aligned}\hat{\pi}_k^{MLE} &= \hat{P}(Y = k) = \frac{N_k}{N} \\ \hat{\theta}_{i,v,k}^{MLE} &= \hat{P}(X_i = v \mid Y = k) = \frac{N_{i,v,k}}{N_k} \\ \hat{P}(\mathbf{X} = \mathbf{x} \mid Y = k) &= \prod_{i=1}^d \hat{P}(X_i = x_i \mid Y = k) \\ &= \prod_{i=1}^d \hat{\theta}_{i,x_i,k}^{MLE} = \prod_{i=1}^d \frac{N_{i,x_i,k}}{N_k}\end{aligned}$$

	ID	Sender known	Topic	Contains link	Urgent words	Class
	1	no	promotion	yes	yes	spam
	2	no	promotion	yes	no	spam
	3	no	account	yes	yes	spam
	4	no	work	yes	yes	spam
	5	yes	promotion	yes	no	spam
	6	no	account	no	yes	spam
	7	yes	account	yes	yes	spam
	8	yes	work	no	no	legitimate
	9	yes	account	no	no	legitimate
	10	yes	account	yes	no	legitimate
	11	yes	work	yes	no	legitimate
	12	yes	account	no	yes	legitimate
	13	yes	work	no	yes	legitimate
	14	yes	account	no	no	legitimate

Practice: Naïve Bayes Email Classification

$\mathbf{x}=(\text{sender known=no,topic=work,contains link=no,urgent words=no})$



$$\hat{\pi}_k^{MLE} = \hat{P}(Y = k)$$

$$\hat{\theta}_{i,v,k}^{MLE,Laplace} = \frac{N_{i,v,k} + 1}{N_k + 2}$$

$$\begin{aligned}\hat{P}(\mathbf{X} = \mathbf{x} \mid Y = k) &= \prod_{i=1}^d \hat{P}(X_i = x_i \mid Y = k) \\ &= \prod_{i=1}^d \hat{\theta}_{i,x_i,k}^{MLE} = \prod_{i=1}^d \frac{N_{i,x_i,k}}{N_k}\end{aligned}$$

$$\begin{aligned}\hat{\theta}_{\text{sender-known=no,class=spam}}^{MLE,Laplace} &= \frac{6}{9} \\ \hat{\theta}_{\text{sender-known=no,class=legitimate}}^{MLE,Laplace} &= \frac{1}{9} \\ \hat{\theta}_{\text{topic=work,class=spam}}^{MLE,Laplace} &= \frac{2}{10} \\ \hat{\theta}_{\text{topic=work,class=legitimate}}^{MLE,Laplace} &= \frac{4}{10} \\ \hat{\theta}_{\text{contains-link=no,class=spam}}^{MLE,Laplace} &= \frac{2}{9} \\ \hat{\theta}_{\text{contains-link=no,class=legitimate}}^{MLE,Laplace} &= \frac{5}{9} \\ \hat{\theta}_{\text{urgent-words=no,class=spam}}^{MLE,Laplace} &= \frac{3}{9} \\ \hat{\theta}_{\text{urgent-words=no,class=legitimate}}^{MLE,Laplace} &= \frac{5}{9}\end{aligned}$$

	ID	Sender known	Topic	Contains link	Urgent words	Class
	1	no	promotion	yes	yes	spam
	2	no	promotion	yes	no	spam
	3	no	account	yes	yes	spam
	4	no	work	yes	yes	spam
	5	yes	promotion	yes	no	spam
	6	no	account	no	yes	spam
	7	yes	account	yes	yes	spam
	8	yes	work	no	no	legitimate
	9	yes	account	no	no	legitimate
	10	yes	account	yes	no	legitimate
	11	yes	work	yes	no	legitimate
	12	yes	account	no	yes	legitimate
	13	yes	work	no	yes	legitimate
	14	yes	account	no	no	legitimate

Practice: Naïve Bayes Email Classification

$\mathbf{x}=(\text{sender known=no,topic=work,contains link=no,urgent words=no})$

$$\hat{P}(\mathbf{X} = \mathbf{x} \mid Y = \textit{spam}) = \frac{6}{9} \times \frac{2}{10} \times \frac{2}{9} \times \frac{3}{9} = \frac{4}{405}$$
$$\hat{P}(\mathbf{X} = \mathbf{x} \mid Y = \textit{legitimate}) = \frac{1}{9} \times \frac{4}{10} \times \frac{5}{9} \times \frac{5}{9} = \frac{10}{729}$$



$$\hat{\pi}_k^{MLE} = \hat{P}(Y = k) = \frac{N_k}{N}$$
$$\hat{\theta}_{i,v,k}^{MLE, Laplace} = \frac{N_{i,v,k} + 1}{N_k + 2}$$
$$\hat{P}(\mathbf{X} = \mathbf{x} \mid Y = k) = \prod_{i=1}^d \hat{P}(X_i = x_i \mid Y = k)$$
$$= \prod_{i=1}^d \hat{\theta}_{i,x_i,k}^{MLE} = \prod_{i=1}^d \frac{N_{i,x_i,k}}{N_k}$$

	ID	Sender known	Topic	Contains link	Urgent words	Class
1	no		promotion	yes	yes	spam
2	no		promotion	yes	no	spam
3	no		account	yes	yes	spam
4	no		work	yes	yes	spam
5	yes		promotion	yes	no	spam
6	no		account	no	yes	spam
7	yes		account	yes	yes	spam
8	yes		work	no	no	legitimate
9	yes		account	no	no	legitimate
10	yes		account	yes	no	legitimate
11	yes		work	yes	no	legitimate
12	yes		account	no	yes	legitimate
13	yes		work	no	yes	legitimate
14	yes		account	no	no	legitimate

Practice: Naïve Bayes Email Classification

$\mathbf{x}=(\text{sender known=no,topic=work,contains link=no,urgent words=no})$

$$\hat{P}(\mathbf{X} = \mathbf{x} | Y = \text{spam}) = \frac{6}{9} \times \frac{2}{10} \times \frac{2}{9} \times \frac{3}{9} = \frac{4}{405}$$
$$\hat{P}(\mathbf{X} = \mathbf{x} | Y = \text{legitimate}) = \frac{1}{9} \times \frac{4}{10} \times \frac{5}{9} \times \frac{5}{9} = \frac{10}{729}$$



$$s_{\text{spam}}(\mathbf{x}) = \hat{P}(Y = \text{spam}) \hat{P}(\mathbf{X} = \mathbf{x} | Y = \text{spam})$$
$$= \frac{7}{14} \times \frac{4}{405} \approx 0.00494$$
$$s_{\text{legitimate}}(\mathbf{x}) = \hat{P}(Y = \text{legitimate}) \hat{P}(\mathbf{X} = \mathbf{x} | Y = \text{legitimate})$$
$$= \frac{7}{14} \times \frac{10}{729} \approx 0.00686$$

$$\hat{\pi}_k^{MLE} = \hat{P}(Y = k) = \frac{N_k}{N}$$
$$\hat{\theta}_{i,v,k}^{MLE, Laplace} = \frac{N_{i,v,k} + 1}{N_k + 2}$$
$$\hat{P}(\mathbf{X} = \mathbf{x} | Y = k) = \prod_{i=1}^d \hat{P}(X_i = x_i | Y = k)$$
$$= \prod_{i=1}^d \hat{\theta}_{i,x_i,k}^{MLE} = \prod_{i=1}^d \frac{N_{i,x_i,k}}{N_k}$$



$$\hat{y} = \arg \max_k s_k(\mathbf{x}) = ?$$



	ID	Sender known	Topic	Contains link	Urgent words	Class
	1	no	promotion	yes	yes	spam
	2	no	promotion	yes	no	spam
	3	no	account	yes	yes	spam
	4	no	work	yes	yes	spam
	5	yes	promotion	yes	no	spam
	6	no	account	no	yes	spam
	7	yes	account	yes	yes	spam
	8	yes	work	no	no	legitimate
	9	yes	account	no	no	legitimate
	10	yes	account	yes	no	legitimate
	11	yes	work	yes	no	legitimate
	12	yes	account	no	yes	legitimate
	13	yes	work	no	yes	legitimate
	14	yes	account	no	no	legitimate

Agenda

- **Decision Tree Learning**
 - **Supervised Classification Problem Setting**
 - **Greedy Top-Down Tree Induction**
 - **Attribute Selection**
 - **Entropy**
 - **Information Gain**
 - **Gini Impurity**
 - **Regression Trees**
 - **Model Complexity and Overfitting**
 - **Tree Pruning**

Decision Tree Learning

Supervised Classification

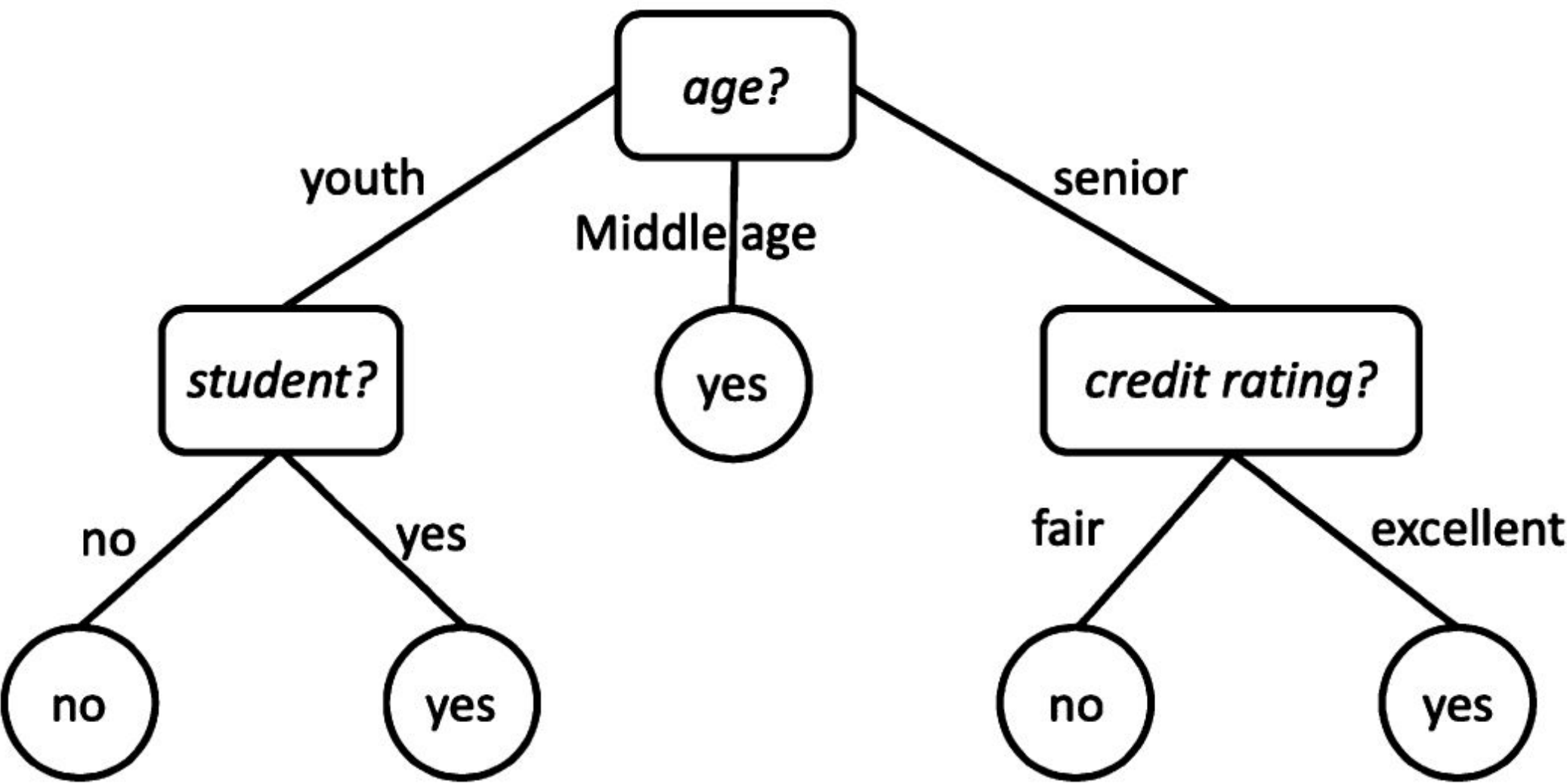
Decision Tree Algorithms: ID3, C4.5, and CART

- In the late 1970s and early 1980s, J. Ross Quinlan developed the **ID3** decision tree algorithm. It was built upon earlier work on concept learning by E. B. Hunt, J. Marin, and P. T. Stone.
- Quinlan later introduced **C4.5**, a successor to ID3. It is a benchmark for comparing new supervised learning algorithms.
- In 1984, L. Breiman, J. Friedman, R. Olshen, and C. Stone published "**Classification and Regression Trees (CART)**".
 - CART describes binary decision tree generation.
- ID3 and CART were developed independently but around the same time. Both ID3 and CART follow a similar approach for learning decision trees. Decision tree induction saw significant growth after these algorithms.
- ID3, C4.5, and CART use a greedy **top-down** approach. Most decision tree algorithms also follow a top-down method starting with a training set of tuples and class labels.

How a Decision Tree Makes Predictions

- Each **internal node**: test one (discrete-valued) attribute X_i
- Each **branch** from a node: corresponds to one possible values for X_i
- Each **leaf node**: predict Y (or $P(Y=1|x \text{ leaf})$)
- The topmost node in a tree is the **root node**

Example: A Decision tree for f : buys computer?



RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no

E.g., $x=(age=youth, student=yes, credit\ rating=excellent)$, $f(x)=Yes$.

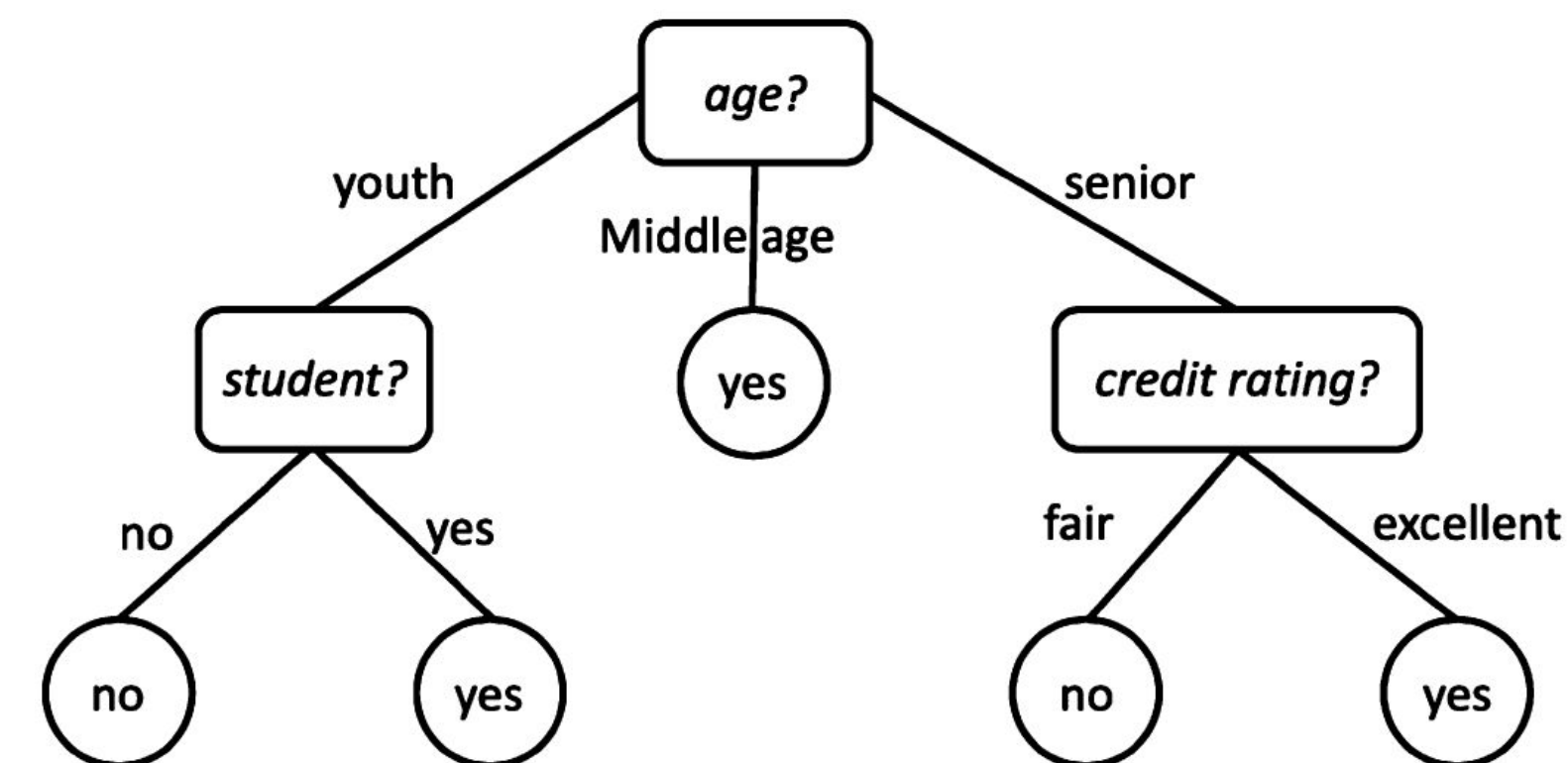
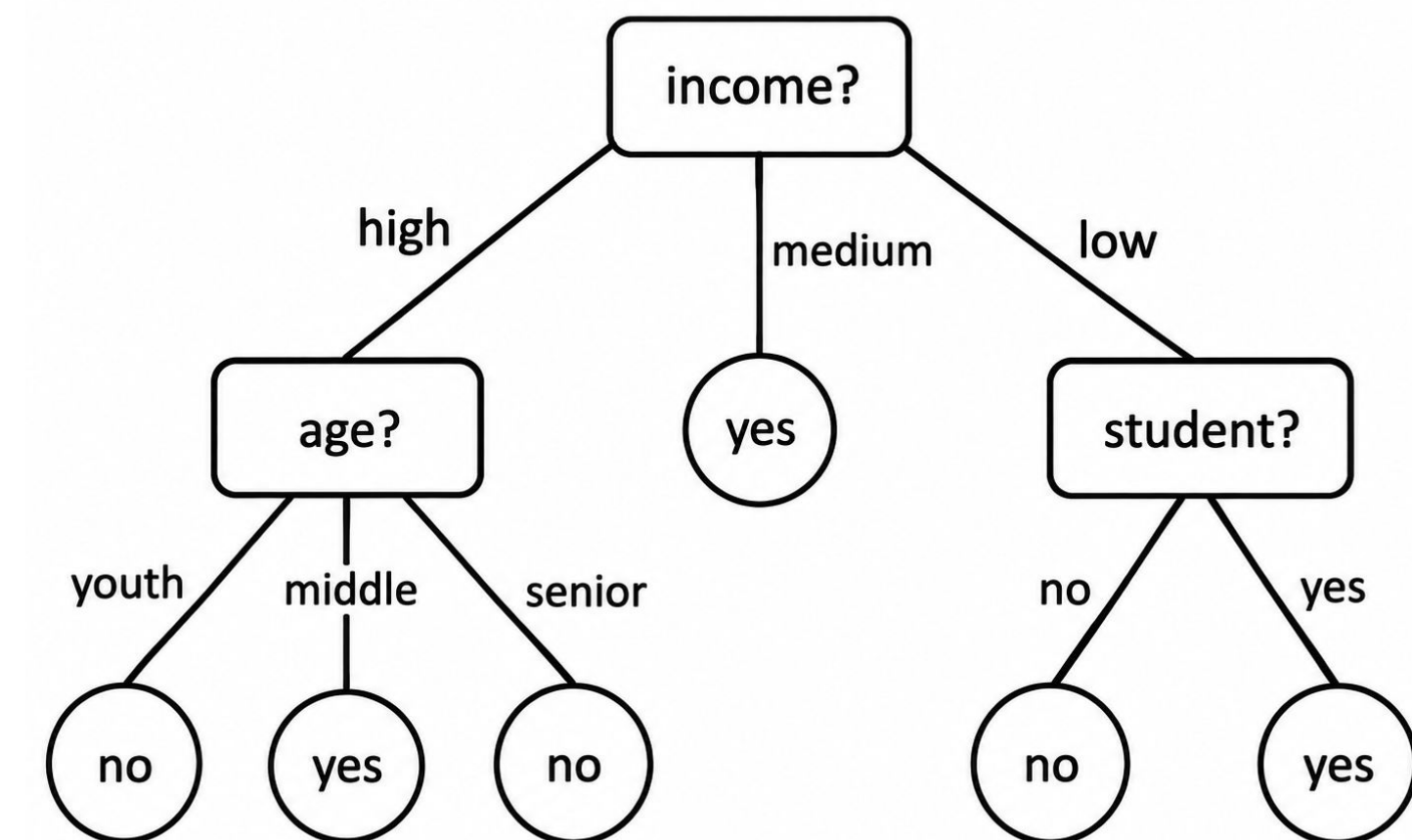
Decision Trees as Hypotheses

Input: Training labeled examples $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ from an unknown target function f :

- Each example is described by its values on a set of **features** or **attributes**.
 - In our example, the four attributes are: age, income, student, credit_rating
 - e.g., $\langle \text{age}=\text{youth}, \text{income}=\text{low}, \text{student}=\text{yes}, \text{credit_rating}=\text{fair} \rangle$
 - The set of all possible input examples is the **instance space**: $\mathbf{x} \in X$
- The unknown target function maps an input example to its class label: $f: X \rightarrow Y$
 - e.g., $Y = \{\text{yes}, \text{no}\}$.
 - For this problem, $f(\mathbf{x}) = \{\text{yes}(\text{customer buys a computer}), \text{no}(\text{customer does not buy a computer})\}$.

Output: A hypothesis $h \in H$ that best approximates the unknown target function f .

- $h: X \rightarrow Y$, where H is the set of possible hypotheses.
 - **Each hypothesis is a decision tree**



Learning and Generalization

- **How to automatically find a good hypothesis for training data?**
 - This is an algorithmic question, the main topic of computer science
- **When do we generalize and do well on unseen data?**
 - Learning theory quantifies ability to generalize as a function of the amount of training data and the hypothesis space
 - Occam's razor: use the simplest hypothesis consistent with data!

Fewer short hypotheses than long ones

- *a short hypothesis that fits the data is less likely to be a statistical coincidence*
- *highly probable that a sufficiently complex hypothesis will fit the data*

Why Greedy Search?

- **How to automatically find a good hypothesis for training data?**
 - This is an algorithmic question, the main topic of computer science
- **When do we generalize and do well on unseen data?**
 - Learning theory quantifies ability to generalize as a function of the amount of training data and the hypothesis space
 - Occam's razor: use the simplest hypothesis consistent with data!
- Decision trees: Smaller trees that fit the data well often have better generalization behavior.
 - NP-hard [Hyafil-Rivest'76]: unlikely to have a poly time algorithm
- Very nice practical heuristics; top down algorithms, e.g, ID3

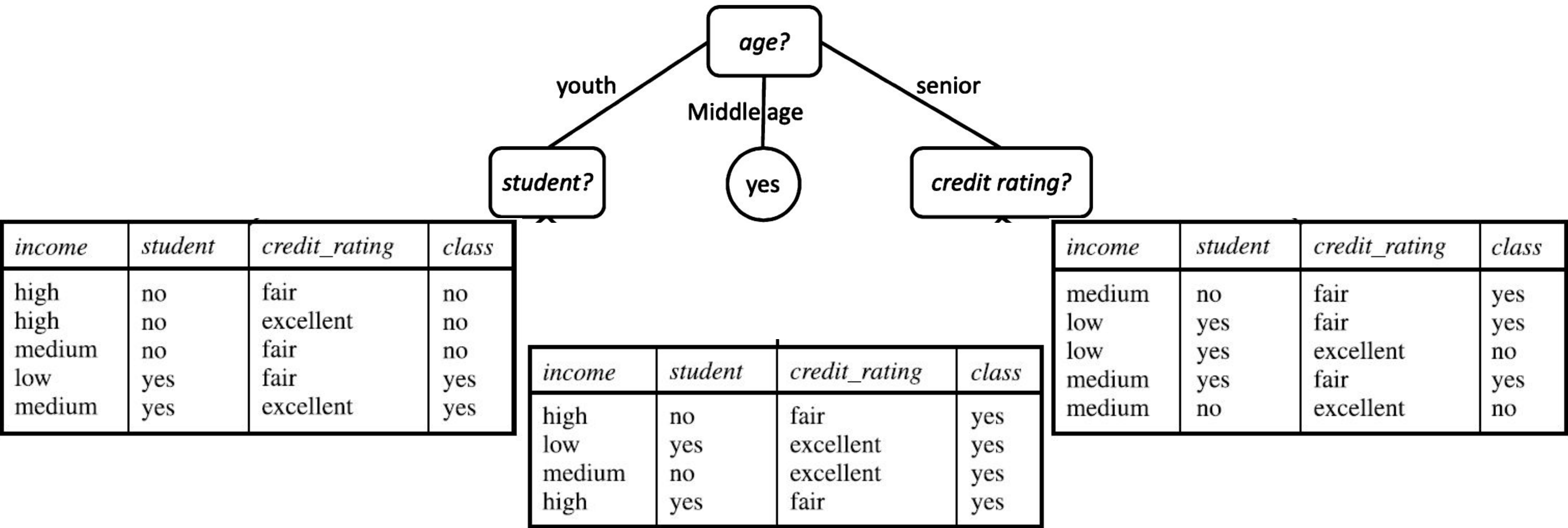


Greedy Top-Down Tree Learning

ID3: Natural greedy approach to growing a decision tree top-down (from the root to the leaves by repeatedly replacing an existing leaf with an internal node.).

Algorithm:

- Pick “best” attribute to split at the root based on training data.
- Recurse on children that are impure (e.g, have both Yes and No).



RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no

Decision Tree Learning Algorithm

ID3: Natural greedy approach to growing a decision tree top-down (from the root to the leaves by repeatedly replacing an existing leaf with an internal node.).

node = Root

Main loop:

1. $A \leftarrow$ the “best” decision attribute for next node
2. Assign A as decision attribute for node
3. For each value of A , create new descendent of node
4. Sort training examples to leaf nodes
5. If training examples perfectly classified, Then STOP, Else iterate over new leaf nodes.



Key question: Which attribute/feature is best?

Choosing the Best Split

An **attribute selection measure** is a heuristic for selecting the splitting criterion that “best” separates a given data partition, D , of class-labeled training tuples into individual classes. If we were to split D into smaller partitions according to the outcomes of the splitting criterion, ideally, each partition would be pure (i.e., all the tuples that fall into a given partition would belong to the same class). Conceptually, the “best” splitting criterion is the one that most closely results in such a scenario. Attribute selection measures are also known as **splitting rules** because they determine how the tuples at a given node are to be split. It provides a ranking for each attribute describing the given training tuples. The attribute having the best score for the measure is chosen as the splitting attribute for the given tuples.

Information Gain: Choosing a Feature



Which attribute/feature is best?

ID3: The attribute with highest information gain.

a statistical measure of how well a given attribute separates the training examples according to the target classification

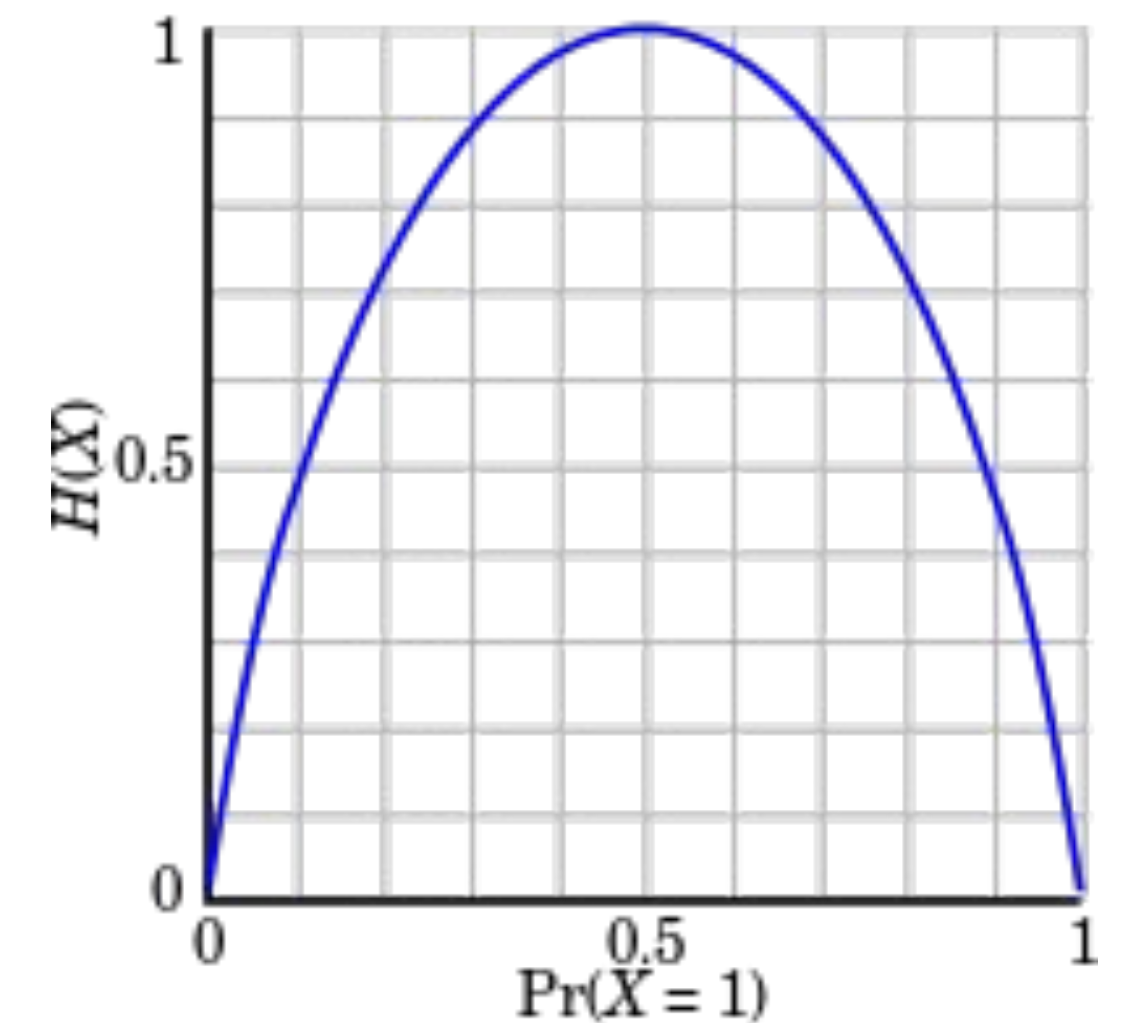
Information Gain of **A** is the expected reduction in entropy of target variable **Y** for data sample **S**, due to sorting on variable **A**

$$\text{Gain}(S, A) = H_S(Y) - H_S(Y|A)$$

Entropy information theoretic measure that characterizes the impurity of a labeled set S .

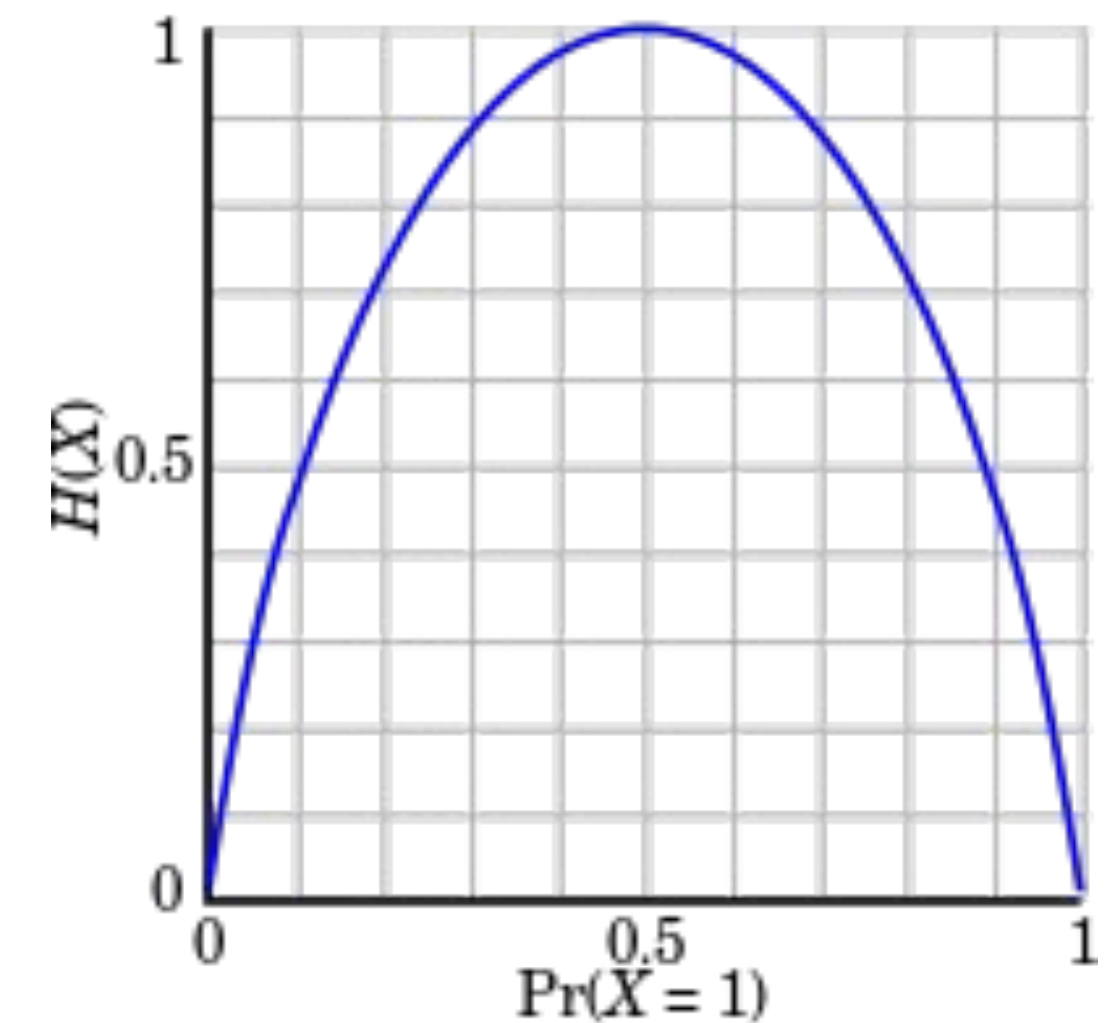
Entropy as a Measure of Impurity

- S is a sample of training examples
 - $p^{\oplus}$ is the proportion of positive examples in S .
 - $p^{\ominus}$ is the proportion of negative examples in S .
 - Entropy measures the impurity of S .
 - $H(S) \equiv -p^{\oplus} \log_2 p^{\oplus} - p^{\ominus} \log_2 p^{\ominus}$
-
- E.g., if all negative, then entropy=0. If all positive, then entropy=0.
 - If 50/50 positive and negative then entropy=1.
 - If 14 examples with 9 positive and 5 negative, then entropy=.940



Interpreting Entropy

- S is a sample of training examples
- $p^{\oplus}$ is the proportion of positive examples in S .
- $p^{\ominus}$ is the proportion of negative examples in S .
- Entropy measures the impurity of S .
 - $H(S) \equiv -p^{\oplus} \log_2 p^{\oplus} - p^{\ominus} \log_2 p^{\ominus}$
 - Higher entropy $\Rightarrow$ higher uncertainty,
 - lower entropy $\Rightarrow$ lower uncertainty.



Interpretation from information theory: expected number of bits needed to encode label of a randomly drawn example in S .

- If S is all positive, receiver knows label will be positive, don't need any bits.
- If S is 50/50 then need 1 bit.
- If S is 80/20, then in a long sequence of messages, can code with less than 1 bit on average (assigning shorter codes to positive examples and longer codes to negative examples).

Information Gain: Reduction in Entropy

Given the definition of entropy, can define a measure of effectiveness of attribute in classifying training data:

Information Gain of **A** is the expected reduction in entropy of target variable **Y** for data sample **S**, due to sorting on variable **A**

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{values}(A)} (|S_v| / |S|) H(S_v)$$

Entropy of original collection

Expected entropy after **S** is partitioned using attribute **A**
sum of entropies of subsets S_v
weighted by the fraction of examples that belong to S_v .

Example: Which Feature Should We Split On?

Example. The following table presents a training set, S, of class-labeled samples randomly selected from the customer database of an electronics store.

buys_computer, has two distinct values (yes, no);
Y(yes) has 9 samples.
Y(no) has 5 samples.

How to choose the (root) node N(splitting attribute) for S?



Try Gain(age)

RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no

- $H(S) \equiv -p^{\oplus} \log_2 p^{\oplus} - p^{\ominus} \log_2 p^{\ominus}$
- $Gain(S, A) = H(S) - \sum_{v \in values(A)} (|S_v| / |S|) H(S_v)$

Computing Gain(S,age)

Example. The following table presents a training set, S, of class-labeled samples randomly selected from the customer database of an electronics store.

buys_computer, has two distinct values (yes, no);
Y(yes) has 9 samples.
Y(no) has 5 samples.

RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	no
					yes
					yes
					yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no

- $H(S) \equiv -p^{\oplus} \log_2 p^{\oplus} - p^{\ominus} \log_2 p^{\ominus}$

$$H(S) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) = 0.940$$

- $Gain(S, A) = H(S) - \sum_{v \in values(A)} (|S_v| / |S|) H(S_v)$

$$\sum_{v \in values(age)} \frac{|S_v|}{|S|} H(S_v) = \frac{5}{14} \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) + \frac{4}{14} \left(-\frac{4}{4} \log_2 \frac{4}{4} \right) + \frac{5}{14} \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right) = 0.694$$

- $Gain(S, Age) = 0.940 - 0.694 = 0.246$

Choosing the Root Feature

Example. The following table presents a training set, S , of class-labeled samples randomly selected from the customer database of an electronics store.

buys_computer, has two distinct values (yes, no);
Y(yes) has 9 samples.
Y(no) has 5 samples.

- $H(S) \equiv -p^{\oplus} \log_2 p^{\oplus} - p^{\ominus} \log_2 p^{\ominus}$
- $Gain(S, A) = H(S) - \sum_{v \in values(A)} (|S_v| / |S|) H(S_v)$
- **$Gain(S, Age) = 0.246$**
- $Gain(S, income) = 0.029$
- $Gain(S, student) = 0.151$
- $Gain(S, credit_rating) = 0.048$

RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no

What About Continuous Features?



If the attribute is a numeric attribute, how can we compute the information gain?

Information Gain for Continuous Features

Suppose, we have an attribute A that is continuous-valued. We must determine the “best” split-point for A , where the split-point is a threshold on A

- We first sort the values of A in the increasing order. Typically, the midpoint between each pair of adjacent values is considered as a possible split-point. Therefore, given v values of A , $(v - 1)$ possible splits are evaluated. For example, the midpoint between the values a_i and a_{i+1} of A is: $(a_i + a_{i+1}) / 2$
- For each possible split-point for A , we evaluate $H_A(S)$, where the number of partitions is two. The point with the minimum expected information requirement for A is selected as the split_point for A .
 - S_1 is the set of tuples in S satisfying $A \leq \text{split_point}$,
 - S_2 is the set of tuples in S satisfying $A > \text{split_point}$.

Gini Impurity and CART

The **Gini impurity**, often referred to as **Gini index(or Gini in short)**, is a measure of impurity or disorder. It is commonly used in decision tree algorithms like CART (Classification and Regression Trees) to determine which feature to split on at any given step in the tree. Given a set D (a data partition or a set of training tuples), the Gini impurity is calculated as:

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2,$$

Where D is the training set, m is the number of unique classes, and p_i is the proportion of instances of class i in D .

If all elements in the set are of the same class (i.e., the set is pure), the Gini impurity will be 0. If the elements are distributed uniformly across different classes, the Gini impurity will be maximized. In the context of decision trees, a lower Gini impurity value indicates that a node is more "pure".

Binary Splits for Categorical Features

Suppose attribute A has v possible values: $A \in \{a_1, a_2, \dots, a_v\}$.

To create a **binary split**, divide the values into two groups:

$$A \in S_A \text{ vs. } A \notin S_A.$$

For example, if $\text{income} \in \{\text{low}, \text{medium}, \text{high}\}$, possible binary splits include:

- $\{\text{low}\}$ vs. $\{\text{medium}, \text{high}\}$
- $\{\text{medium}\}$ vs. $\{\text{low}, \text{high}\}$
- $\{\text{high}\}$ vs. $\{\text{low}, \text{medium}\}$.

The number of unique binary splits is $2^{v-1} - 1$.

Each possible split can be evaluated using an impurity measure such as **Gini impurity**.

Weighted Gini After a Split

When considering a binary split, the goal is to compute a weighted sum of the impurity of each resulting partition. Specifically:

- Let's assume a binary split on attribute A divides dataset D into two subsets D_1 and D_2 .
- The Gini impurity of D after this split is computed as:

$$Gini_A(D) = \frac{|D_1|}{|D|} \times Gini(D_1) + \frac{|D_2|}{|D|} \times Gini(D_2)$$

For discrete-valued attributes:

- Every possible binary split is evaluated.
- The split that minimizes the Gini impurity for that attribute is chosen as the best split.

Gini Splits for Continuous Features

Continuous-valued Attributes and Split-Points

For continuous-valued attributes, the task is to evaluate every possible split-point. This approach mirrors the method previously outlined for information gain. Specifically:

- We sort the values of the attribute.
- For each pair of adjacent values, the midpoint serves as a potential split-point.

The optimal split-point is the one which minimizes the Gini impurity for the continuous-valued attribute in question. To elucidate:

- For a potential split-point of attribute A , subset D_1 comprises tuples in D where $A \leq \text{split_point}$.
- Conversely, D_2 contains tuples in D where $A > \text{split_point}$.

Example: Choosing a Split with Gini

Example. Consider the training data in the following table, denoted as D . In this data, nine tuples are classified under the category `buys_computer = yes`, while the remaining five are categorized as `buys_computer = no`. We initiate by creating a root node, N , representing the tuples in D . How to compute N using Gini impurity?

RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no



$$Gini(D) = 1 - \sum_{i=1}^m p_i^2,$$
$$Gini_A(D) = \frac{|D_1|}{|D|} \times Gini(D_1) + \frac{|D_2|}{|D|} \times Gini(D_2)$$

Computing Gini

To identify the optimal splitting criterion for D , we evaluate the Gini impurity for each attribute.

Example. Consider the training data in the following table, denoted as D . In this data, nine tuples are classified under the category `buys_computer = yes`, while the remaining five are categorized as `buys_computer = no`. We initiate by creating a root node, N , representing the tuples in D . How to compute N using Gini impurity?

RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no

- Other subsets:
 - $Gini_{income \in \{low, high\}}(D) = 0.458$
 - $Gini_{income \in \{medium, high\}}(D) = 0.450$

Income Attribute:

- Subset {low, medium}:

$$\begin{aligned} &Gini_{income \in \{low, medium\}}(D) \\ &= \frac{10}{14} \left(1 - \left(\frac{7}{10} \right)^2 - \left(\frac{3}{10} \right)^2 \right) \\ &+ \frac{4}{14} \left(1 - \left(\frac{2}{4} \right)^2 - \left(\frac{2}{4} \right)^2 \right) = 0.443 \end{aligned}$$

$$Gini_A(D) = \frac{|D_1|}{|D|} \times Gini(D_1) + \frac{|D_2|}{|D|} \times Gini(D_2)$$

Choosing the Best Split

To identify the optimal splitting criterion for D , we evaluate the Gini impurity for each attribute.

Example. Consider the training data in the following table, denoted as D . In this data, nine tuples are classified under the category `buys_computer = yes`, while the remaining five are categorized as `buys_computer = no`. We initiate by creating a root node, N , representing the tuples in D . How to compute N using Gini impurity?

RID	age	income	student	credit_rating	buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle	medium	no	excellent	yes
13	middle	high	yes	fair	yes
14	senior	medium	no	excellent	no

- `income`: Best split is {low, medium} or {high} with a Gini impurity of 0.443
- `age`: Best split is {youth, senior} or {middle_aged} with a Gini impurity of 0.357. **win!!!!!!**
- `student`: Binary attributes with Gini impurity values of 0.367
- `credit_rating`: Binary attributes with Gini impurity values of 0.429

$$Gini_A(D) = \frac{|D_1|}{|D|} \times Gini(D_1) + \frac{|D_2|}{|D|} \times Gini(D_2)$$

Information Gain vs. Gini Impurity

Criteria	Information Gain	Gini Index
Purpose	Measures the impurity based on the average amount of information needed to identify the class label.	Quantifies the likelihood of mis-classification of a randomly chosen tuple based on class label distribution.
Theory	Information theory.	Based on mis-classification.
Split	ID3 commonly uses multiway splits for categorical features	CART commonly uses binary splits.
Efficiency	Involves logarithm computation, making it slightly less efficient.	More computationally efficient
Outcome	Both measures often lead to very similar decision trees.	

Regression Trees

Regression tree is used to predict the continuous output value.

- Similar to a decision tree in that it also partitions the entire attribute space into multiple subregions, each corresponding to a leaf node.
- The main difference:
 - A leaf node holds a continuous value instead of a categorical value (i.e., class label) in a decision tree. The continuous value of a leaf node is learned during the training phase, which is set as the average output value of all training tuples fallen in the corresponding subregions. “Classification and Regression Trees” (CART) uses residual sum of squares (RSS) as the objective function to train a regression tree.

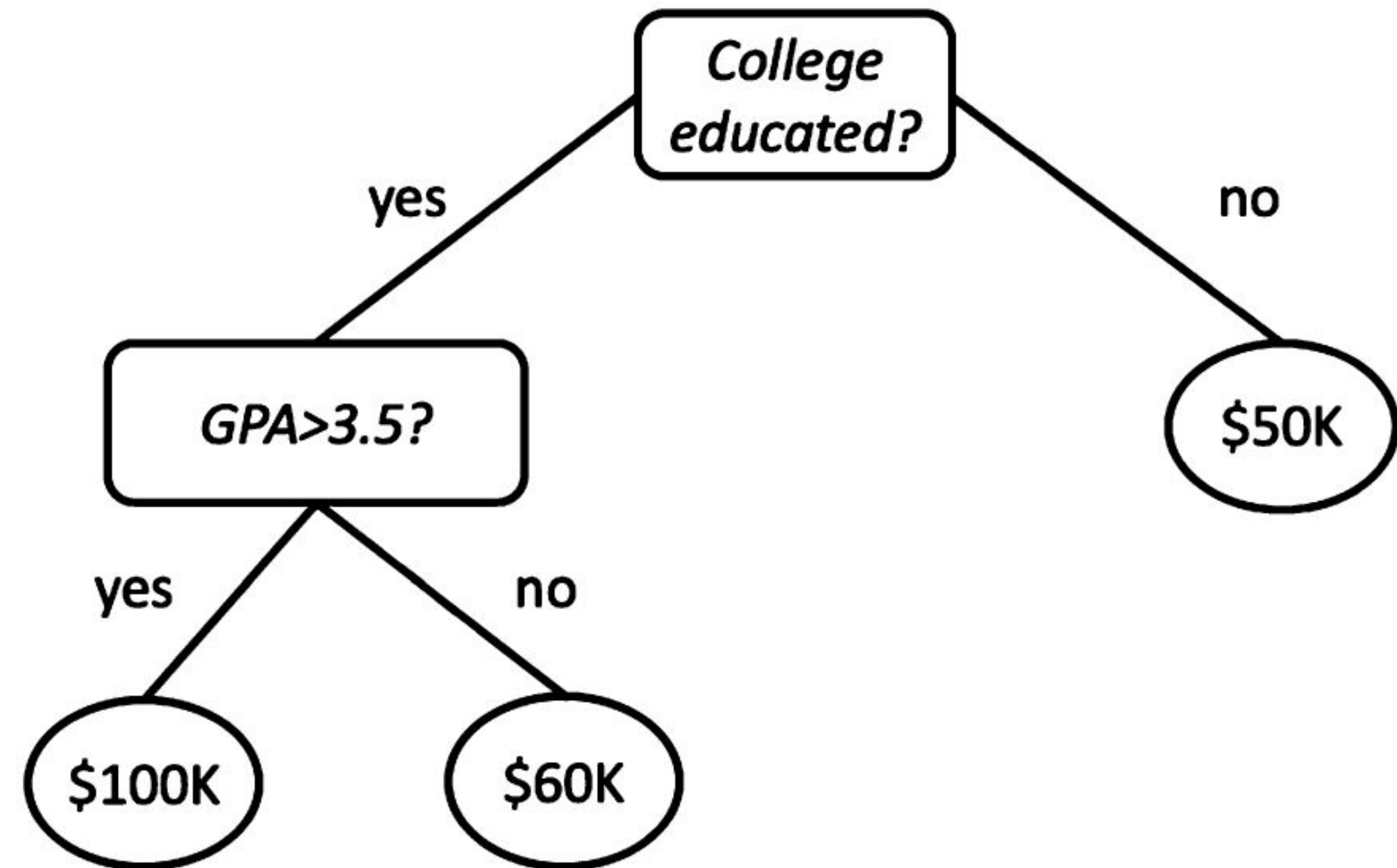
$$RSS = \sum_i (y_i - \hat{y}_i)^2$$

where y_i is the actual output value of the i th training tuple, and $\hat{y}_i$ is the predicted output by the regression tree.

How Regression Trees Make Predictions

Example: A regression tree for predicting the average yearly income based on an individual's education.

\$50K is the average yearly income of all training individuals who do not have a college degree; \$60K is the average yearly income of all training individuals who have a college degree with a GPA less than or equal to 3.5; and \$100K is the average yearly income of all training individuals who have a college degree with a GPA higher than 3.5. The leaf node values (\$50K, \$60K, and \$100K) are used to predict the yearly income of any test individual who falls into the corresponding leaf nodes.



Example: Regression Tree Splits Using RSS

Example: Consider a regression tree node containing five training examples.

The input feature x_i is **years of experience**, and the target y_i is **annual salary (in \$K)**.

Our goal is to find the best split point for years of experience that divides the data into two child nodes.



				Employee	Years of experience (x_i)	Annual salary (y_i) (\$K)
$RSS = \sum_i (y_i - \hat{y}_i)^2$				1	1.2	45
				2	2.4	48
				3	3.1	52
				4	5.8	70
				5	7.0	74

Candidate Split Points

Example: Consider a regression tree node containing five training examples.

The input feature x_i is **years of experience**, and the target y_i is **annual salary (in \$K)**.

Our goal is to find the best split point for years of experience that divides the data into two child nodes.

$$\frac{1.2+2.4}{2} = 1.8$$

Candidate split points: **[1.8, 2.75, 4.45, 6.4]**

$$\frac{2.4+3.1}{2} = 2.75$$

$$\frac{3.1+5.8}{2} = 4.45$$

$$\frac{5.8+7.0}{2} = 6.4$$

$$RSS = \sum_i (y_i - \hat{y}_i)^2$$

Employee	Years of experience (xi)	Annual salary (yi) (\$K)
1	1.2	45
2	2.4	48
3	3.1	52
4	5.8	70
5	7.0	74

Computing RSS for One Split

Example: Consider a regression tree node containing five training examples.

The input feature x_i is **years of experience**, and the target y_i is **annual salary (in \$K)**.

Our goal is to find the best split point for years of experience that divides the data into two child nodes.

Candidate split points: **[1.8, 2.75, 4.45, 6.4]**

Choose 1.8  $y_{\text{left}} = 45$, $y_{\text{right}} = (48 + 52 + 70 + 74) / 4 = 61$



$$\text{RSS}_{1.8} = (y_1 - y_{\text{left}})^2 + (y_2 - y_{\text{right}})^2 + (y_3 - y_{\text{right}})^2 + (y_4 - y_{\text{right}})^2 + (y_5 - y_{\text{right}})^2 = 500$$

Employee	Years of experience (xi)	Annual salary (yi) (\$K)
1	1.2	45
2	2.4	48
3	3.1	52
4	5.8	70
5	7.0	74

$$RSS = \sum_i (y_i - \hat{y}_i)^2$$

Choosing the Best RSS Split

Example: Consider a regression tree node containing five training examples.

The input feature x_i is **years of experience**, and the target y_i is **annual salary (in \$K)**.

Our goal is to find the best split point for years of experience that divides the data into two child nodes.

Candidate split points: [1.8, 2.75, 4.45, 6.4]

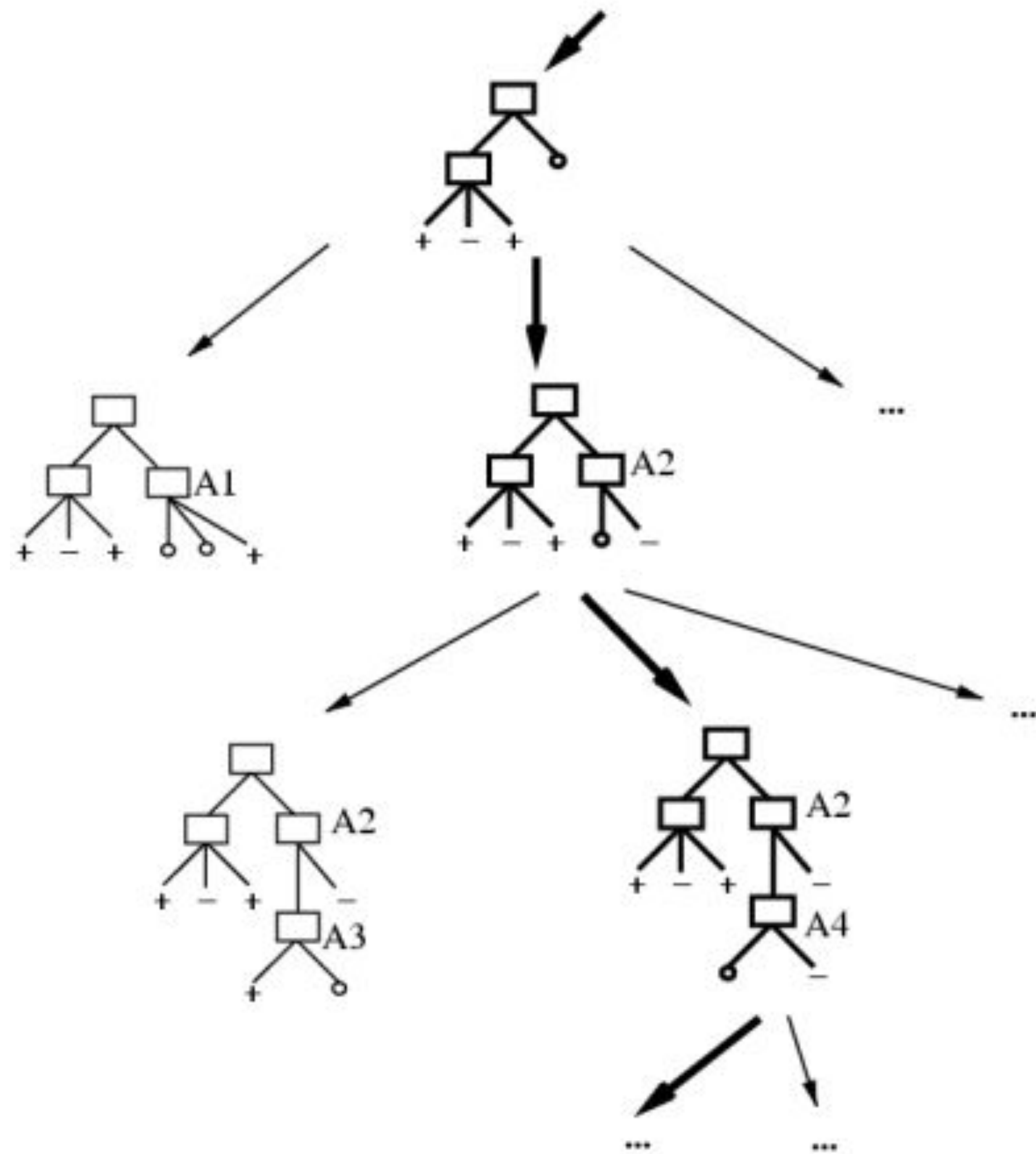
split point	y_{left}	y_{right}	RSS
1.8	45.00	61.00	500.00
2.75	46.50	65.33	279.17
4.45	48.33	72.00	32.67
6.4	53.75	74.00	376.75



Employee	Years of experience (x_i)	Annual salary (y_i) (\$K)
1	1.2	45
2	2.4	48
3	3.1	52
4	5.8	70
5	7.0	74

$$RSS = \sum_i (y_i - \hat{y}_i)^2$$

Searching for a Good Decision Tree

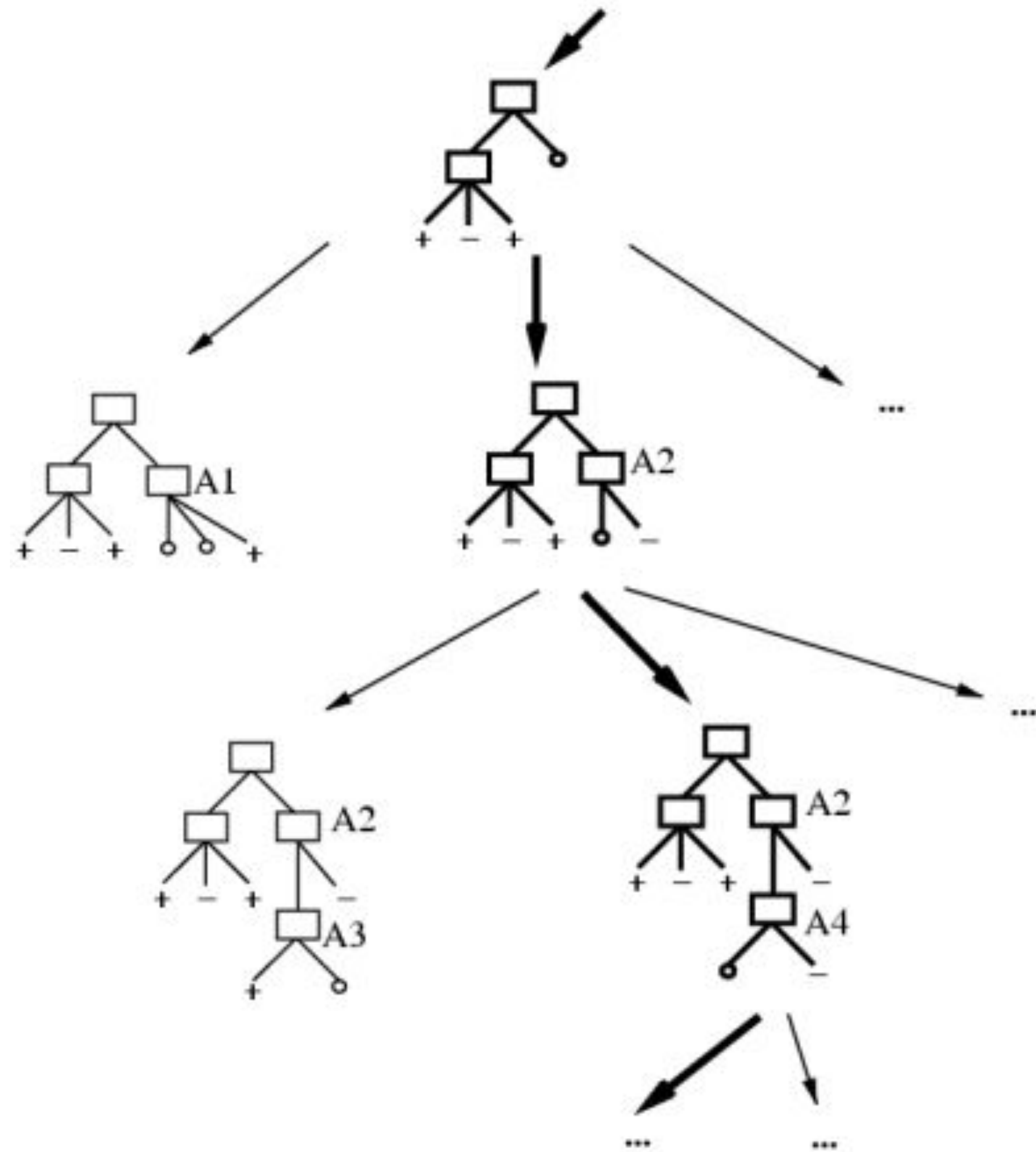


- ID3 performs heuristic search through space of decision trees
- ID3 greedily searches the space of decision trees. It has a preference for relatively simple trees, but does not guarantee the globally smallest tree.



Occam's razor: prefer the simplest hypothesis that fits the data

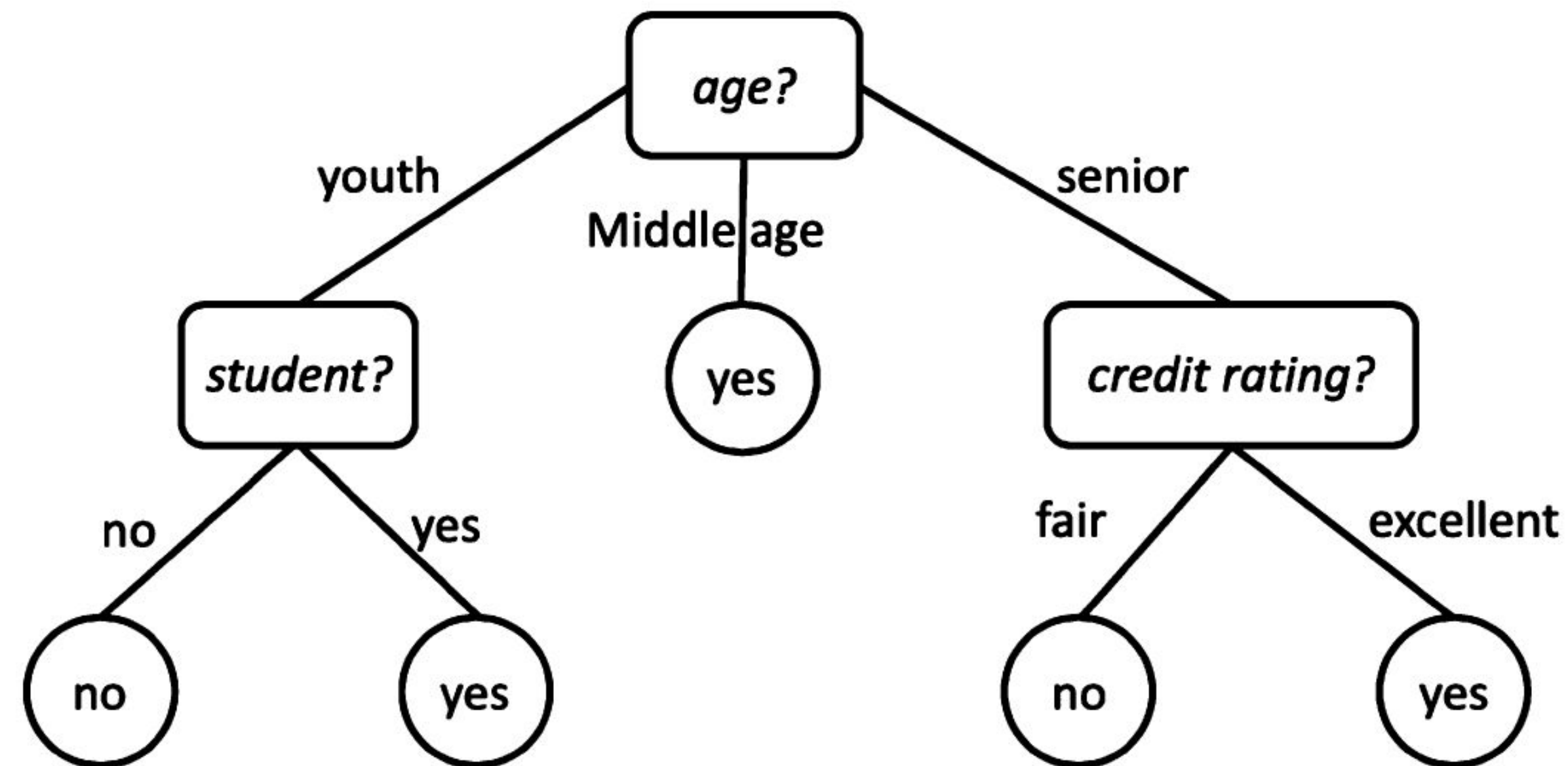
Tree Complexity and Overfitting



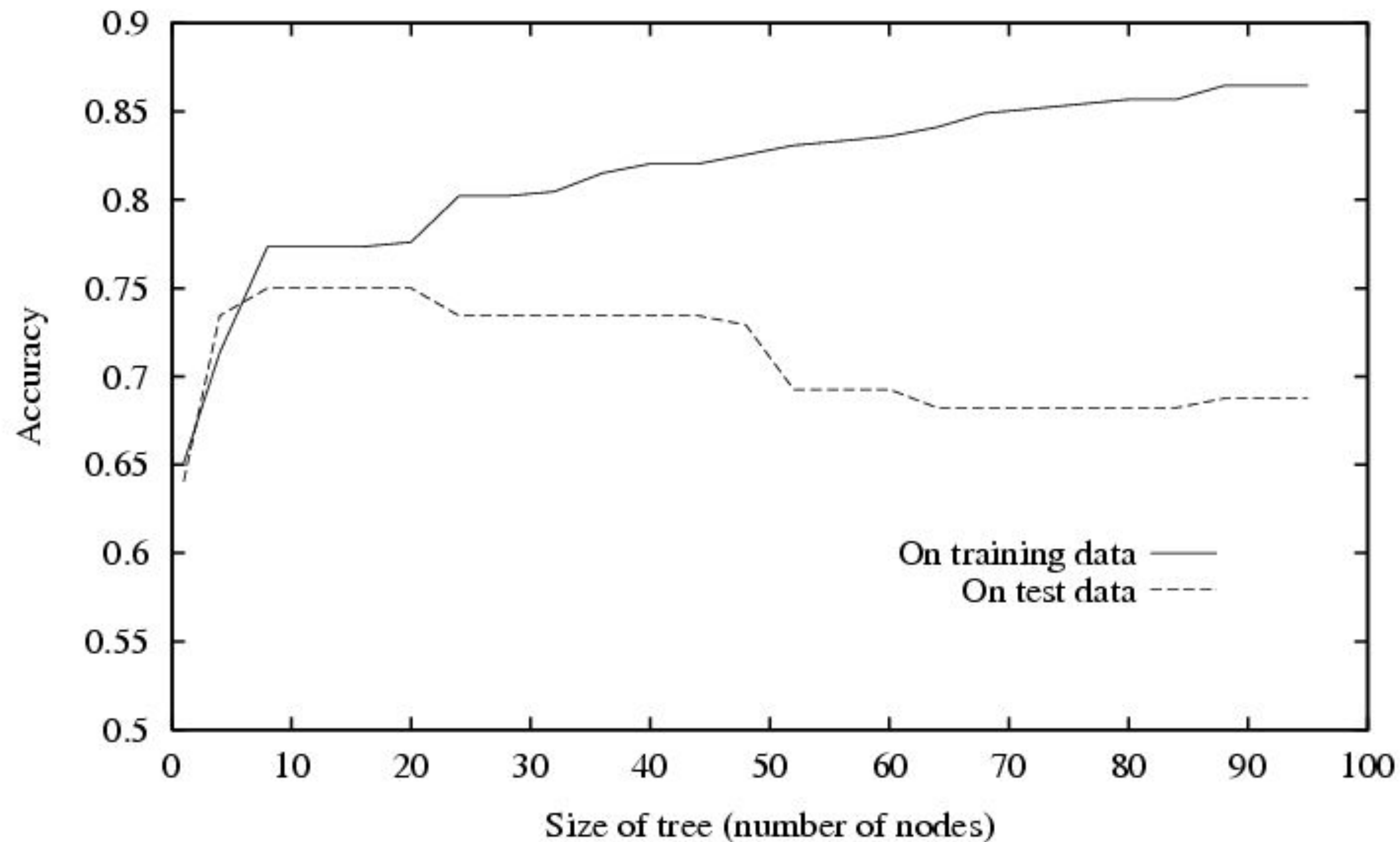
- ID3 performs heuristic search through space of decision trees
 - It prefers relatively simple trees, but a tree can still become too complex and overfit the training data.
 - It might be beneficial to prune the tree by using a validation dataset.

How Noise Can Change a Decision Tree

- Consider adding noisy training example:
age = youth, income = high, student = yes, credit_rating = fair, buys_computer = no
- What effect on earlier tree?



Tree Size and Generalization



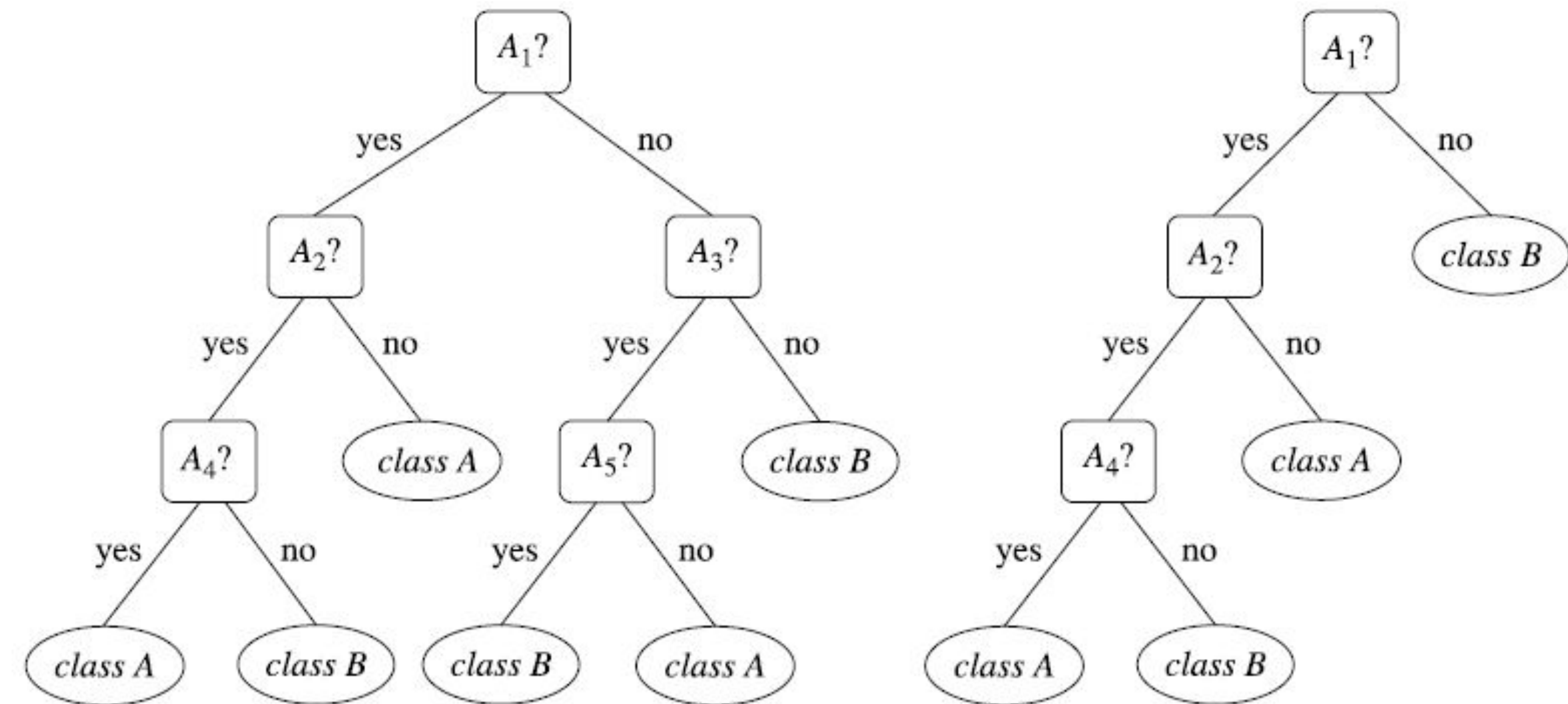
How can we avoid overfitting?

Avoiding Overfitting with Pruning

Decision trees, when constructed, can sometimes become overly complex, capturing anomalies in the training data that result from noise or outliers. Such complexity can adversely affect the accuracy of predictions for new, unseen samples (overfitting). **Tree pruning** addresses this overfitting issue by eliminating branches that don't contribute significant power in predicting the outcome.

Benefits of Pruned Trees:

- **Simplicity:** They are smaller, simpler and easier to understand.
- **Speed:** Faster to execute due to reduced size.
- **Accuracy:** Pruning can improve generalization by reducing overfitting.



Prepruning vs. Postpruning

Two approaches:

- **Prepruning:** The construction of the tree is halted early. This can be done by setting criteria like a minimum information gain or a specific level of statistical significance. If a potential split in the tree doesn't meet these criteria, the tree's growth is stopped at that point, turning the node into a leaf. This leaf will then either represent the most frequent class among the data at that node or show the probability distribution of the classes.
 - Difficult to choose an appropriate threshold:
 - High thresholds => oversimplified trees, low thresholds => very little simplification.
- **Postpruning:** Trimming subtrees from a fully developed tree. In this process, a subtree rooted at a particular node is removed, and the node is transformed into a **leaf**. This new leaf is then labeled with the dominant class label from the pruned subtree. **more common approach**

Summary

- A decision tree represents a hypothesis for supervised learning.
- Decision trees are learned using a greedy top-down procedure.
- At each node, the algorithm selects a split that improves class purity.
- Common split criteria include:
 - Information Gain
 - Gini Impurity
- Continuous features can be split using threshold values.
- Regression trees predict continuous outputs and commonly use RSS to choose splits.
- Decision trees can become too complex and overfit the training data.
- Pruning and other complexity-control methods can improve generalization.